VIB Knowledge Assistant

Giải thích kiến trúc hệ thống bằng ngôn ngữ dễ hiểu

COS40005 Computing Technology Project A · Swinburne University of Technology · Khách hàng: VIB (qua chương trình AWS FCAJ)

> **Tài liệu này dùng để làm gì?**
> Giúp bạn nắm chắc hệ thống trước khi trình bày với anh Phát và giảng viên. Dựa trên Charter, Proposal, Team Plan, biên bản họp 22/10/2026 và bản nháp draw.io của nhóm. Điểm nào là **đề xuất của mình** (chưa phải quyết định của nhóm) đều ghi rõ là "đề xuất".

1. Hệ thống làm gì, trong một đoạn

Nhân viên hỏi bot ngay trong **Slack**. Bot kiểm tra người hỏi có quyền không, tìm câu trả lời trong tài liệu **Confluence** của công ty, viết câu trả lời ngắn gọn và **gắn link đúng trang nguồn**. Bot trả lời bằng tiếng Anh hoặc tiếng Việt theo ngôn ngữ của câu hỏi.

Vấn đề cần giải quyết

- Kiến thức công ty nằm rải rác: các không gian Confluence (Products, Legal, Policies, Benefits), trang tài liệu, repository, diễn đàn.
- Nhân viên mất thời gian tìm, phải rời Slack, và hay hỏi lại đồng nghiệp cùng một câu.
- Công cụ tìm kiếm thông thường không cho biết người hỏi có được phép xem tài liệu đó hay không.

Mục tiêu đo được (theo Project Charter)

Mục tiêu	Chỉ tiêu
Độ chính xác câu trả lời	Ít nhất **80%** trên bộ câu hỏi song ngữ Anh–Việt, tính riêng từng ngôn ngữ
Tốc độ	Trung bình **dưới 3 giây** cho mỗi lượt hỏi trên Slack
Tuân thủ whitelist	**100%**: không trả lời ngoài các kênh đã duyệt
Tiết kiệm thời gian tìm kiếm	Khoảng **40%** trong giai đoạn đánh giá thử
Chi phí	**Không phát sinh thêm** trong giai đoạn proof of concept (Alpha)

Tiến độ: từ 29/09/2026 đến 06/12/2026. Bản Alpha (proof of concept) làm trong Project A (tuần 5–12). Tài liệu đặc tả kỹ thuật gửi anh Phát ngày 02/12/2026, bàn giao ngày 06/12/2026.

2. RAG là gì? (giải thích như đi thi mở sách)

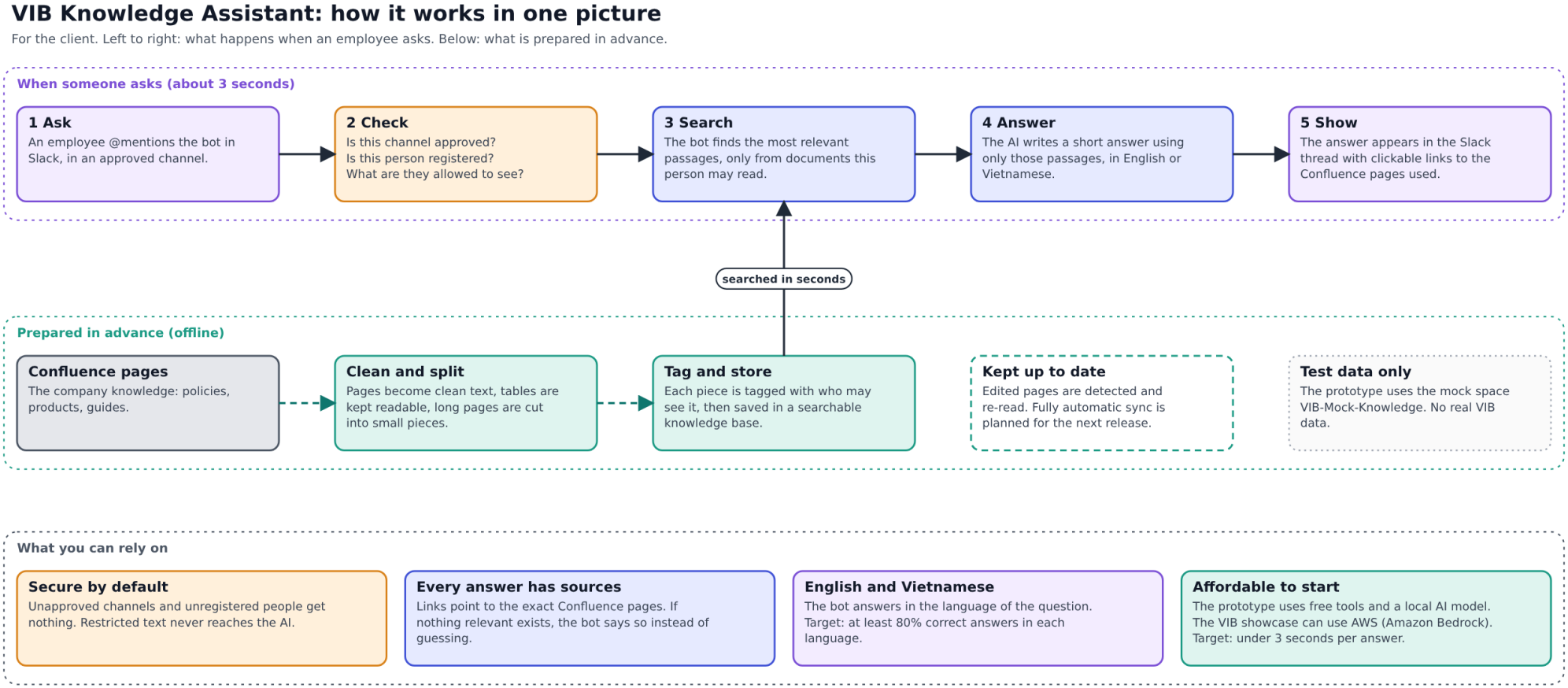
RAG viết tắt của *Retrieval-Augmented Generation*, nghĩa là "**tìm trước, viết sau**". Hãy hình dung một bài thi được mở sách:

- **Tìm (Retrieval):** người làm bài không cố nhớ mọi thứ, mà lật sách để tìm vài đoạn liên quan nhất.
- **Viết (Generation):** rồi viết câu trả lời dựa trên đúng những đoạn đó và ghi rõ lấy từ trang nào.

Vì sao không hỏi thẳng một AI? Vì AI không biết tài liệu nội bộ của VIB và dễ **nói bịa** khi không có tài liệu trước mặt. Nhóm cũng đã loại phương án mua sản phẩm tìm kiếm có sẵn (Slack AI, Atlassian Rovo) vì tốn phí và khó tùy biến.

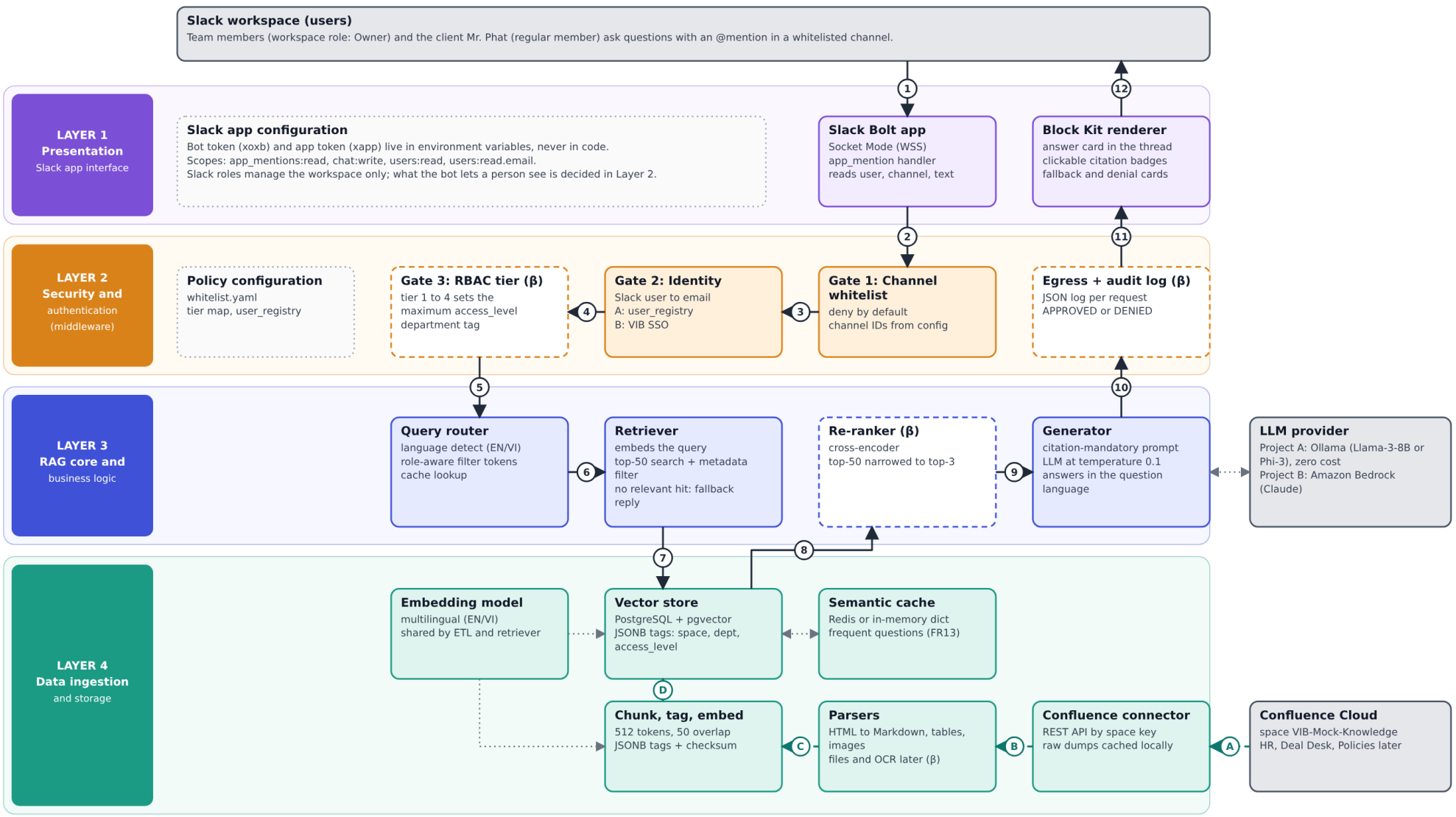
3. Bức tranh tổng thể

Hình dưới là bản tóm tắt dành cho khách hàng: phía trên là điều xảy ra khi có người hỏi (khoảng 3 giây), phía dưới là phần chuẩn bị sẵn từ trước.



Hình 1. Tóm tắt một trang cho client.

Kiến trúc đầy đủ gồm 4 lớp



Hình 2. Kiến trúc tổng thể. Mũi tên liền có số (1–12) là đường hỏi–đáp. Mũi tên nét đứt có chữ (A–D) là đường nạp dữ liệu. Ô viền nét đứt là tính năng Beta.

Cách đọc Hình 2

- Đọc **từ trên xuống** theo số 1 → 12: câu hỏi đi xuống qua các lớp, câu trả lời đi ngược lên.
- Mỗi lớp có **một màu** riêng (tím, cam, xanh dương, xanh lá) và dùng cùng màu ở các hình chi tiết phía sau.

Đối chiếu với bản nháp của nhóm
Bản nháp draw.io chia theo Zone 1–5. Mình xếp lại thành 4 lớp cho khớp hình tham chiếu của giảng viên: Zone 5 = Presentation, Zone 4 = Security, Zone 3 = RAG core, Zone 1–2 = Data ingestion and storage.

4. Bốn lớp chi tiết

Lớp	Làm gì	Ví dụ đời thường
1. Presentation	Nhận @mention trong Slack, hiển thị câu trả lời kèm nút mở trang nguồn	Quầy lễ tân
2. Security	Kiểm tra kênh, người hỏi và quyền xem. Sai một bước là dừng	Cổng bảo vệ
3. RAG core	Hiểu câu hỏi, tìm đoạn phù hợp, viết câu trả lời có dẫn nguồn	Người nghiên cứu
4. Data ingestion and storage	Đọc Confluence, làm sạch, chia nhỏ, gắn nhãn, lưu thành kho dễ tìm	Thư viện và sổ mục lục

Lớp 1 · Presentation (giao diện Slack)

Vai trò: nhận câu hỏi và hiển thị câu trả lời, **không chứa logic nghiệp vụ**.

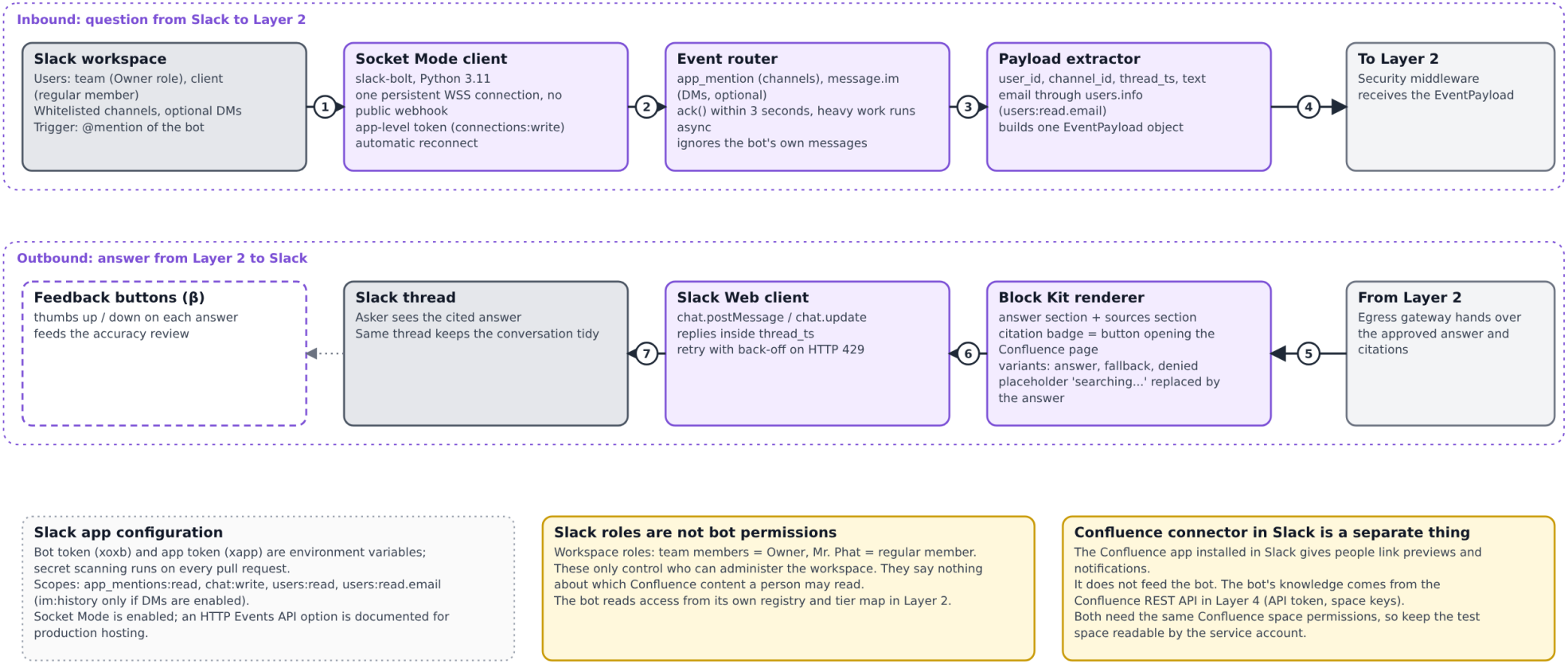
Thành phần	Việc làm	Công nghệ
Socket Mode client	Giữ một kết nối WebSocket lâu dài tới Slack, tự kết nối lại khi rớt	slack-bolt (Python 3.11), app token xapp
Event router	Lắng nghe sự kiện app_mention. Phải phản hồi ack() trong 3 giây, phần xử lý nặng chạy ngầm. Bỏ qua tin nhắn của chính bot	Bolt event handler
Payload extractor	Lấy user_id, channel_id, thread_ts, nội dung và email của người hỏi (qua users.info) thành một gói EventPayload gửi xuống Lớp 2	Scope users:read.email
Block Kit renderer và Web client	Dựng thẻ trả lời (câu trả lời, nguồn, nút trích dẫn mở trang Confluence; có bản dự phòng và từ chối) rồi đăng vào đúng thread bằng chat.postMessage, thử lại khi gặp HTTP 429	Slack Block Kit, WebClient

Vì sao dùng Socket Mode? Bot tự kết nối *ra* Slack nên lúc thử nghiệm không cần địa chỉ công khai hay SSL. Bản triển khai thật có thể dùng HTTP Events API.

Hai chỗ rất dễ nhầm
1. Role Slack không phải quyền của bot. Owner hay thành viên thường chỉ quyết định ai quản trị workspace. Ai được xem tài liệu nào do bảng người dùng của bot (Lớp 2) quyết định.
2. "Confluence connector" cài trong Slack chỉ để xem trước link và nhận thông báo, không cấp dữ liệu cho bot. Bot đọc Confluence bằng REST API với API token (Lớp 4).

Layer 1: Presentation (Slack app interface)

Receives the @mention, turns it into a clean event payload, and renders the final answer. No business logic lives here.



Hình 3. Chi tiết Lớp 1: luồng vào (1–4), luồng ra (5–7) và các ghi chú về cấu hình Slack.

Lớp 2 · Security (bảo mật và xác thực)

Nguyên tắc: zero-trust và **deny by default** (mặc định là từ chối). Có ba cổng theo thứ tự cố định. Rớt một cổng là dừng: không có gì được tìm kiếm và không có gì được gửi cho AI.

Cổng	Câu hỏi nó đặt ra	Cách làm
Cổng 1: Channel whitelist	Tin nhắn có đến từ kênh được duyệt không?	So channel_id với file cấu hình whitelist.yaml. Kênh lạ thì bỏ qua. Tin nhắn riêng (DM) chỉ cho phép khi bật rõ ràng
Cổng 2: Identity	Người này là ai và đã được duyệt chưa?	**Project A:** bảng user_registry trong PostgreSQL do Admin duyệt (vì nhóm không có email VIB, dùng email thường vẫn được). **Project B:** đăng nhập một lần của VIB (SSO) và kiểm tra tên miền email @vib.com.vn
Cổng 3: Tầng quyền (Beta)	Người này được xem mức tài liệu nào?	Tra tầng 1–4 và phòng ban, trả về security_context

Sau cổng 3, hệ thống tạo **security_context** gồm user_id, email, tier, dept và request_id, rồi chuyển cho Lớp 3 cùng với câu hỏi.

Bảng tầng quyền (thiết kế của nhóm)

Tầng	Ai	Được xem	Bộ lọc trong truy vấn SQL
1	Nhân viên	Tài liệu công khai và nội bộ chung	access_level ≤ 1
2	Quản lý / Deal Desk	Tài liệu nội bộ phòng ban, bảng giá	access_level ≤ 2 và department_tag = phòng ban của người hỏi
3	Lãnh đạo	Tài liệu liên phòng ban, tóm tắt tài chính hạn chế	access_level ≤ 3 và vai trò là Quản lý hoặc Lãnh đạo
4	Quản trị hệ thống	Vận hành pipeline, xem audit log, gán quyền	Dữ liệu hạn chế cao còn cần token cấp phép của admin

Điểm quan trọng nhất của lớp này
Bộ lọc quyền được áp dụng **ngay bên trong câu truy vấn cơ sở dữ liệu**, tức là trước khi AI nhìn thấy bất kỳ đoạn văn nào. Không có cách "lọc sau khi AI viết xong", vì lúc đó thông tin đã bị lộ vào câu trả lời. Lớp 2 còn kiểm tra thêm một lần ở cuối (cổng ra) rằng mọi tài liệu được trích dẫn đều nằm trong quyền của người hỏi.

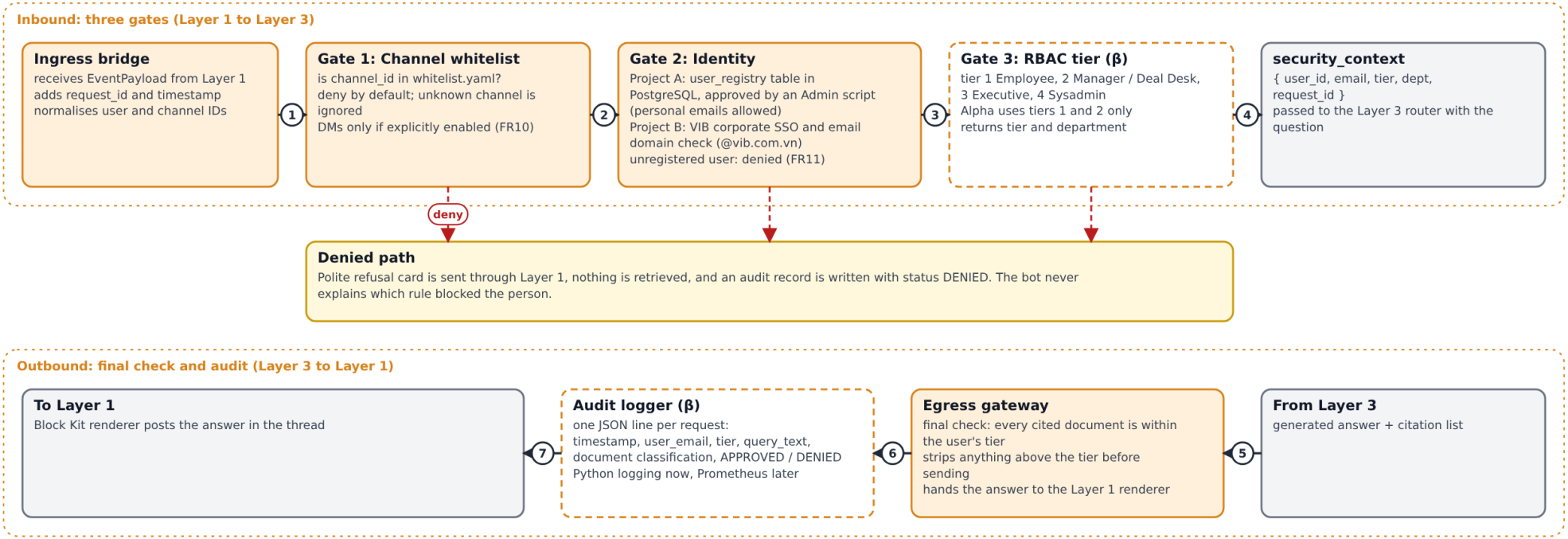
Đề xuất ánh xạ role Slack sang tầng quyền (đề xuất, nhóm cần xác nhận)

- Role Slack **không** dùng để cấp quyền. Bảng đăng ký (registry) mới quyết định.
- Các thành viên nhóm (Owner trong Slack): Admin đăng ký bằng script. Project manager = tầng 4, các bạn dev = tầng 2 để thử các trang bị hạn chế.
- Anh Phát (thành viên thường trong Slack): tầng 1, để anh thấy đúng điều một nhân viên bình thường sẽ thấy.
- Ai không có trong registry, kể cả Owner: bị từ chối. Ở Project B chỉ cần đổi bước tra registry thành SSO của VIB, bảng tầng giữ nguyên.

Audit (Beta): mỗi yêu cầu ghi một dòng JSON: thời gian, email, tầng, câu hỏi, loại tài liệu, APPROVED hoặc DENIED.

Layer 2: Security and authentication (middleware)

Three gates in a fixed order. Any failed gate stops the request: nothing reaches retrieval or the LLM. Zero-trust, deny by default.



Tier table (team design, access_level is stamped on every chunk at ingestion)

Tier 1 Employee: public and internal documents. Filter: access_level <= 1.
Tier 2 Manager / Deal Desk lead: department internal files, pricing guides. Filter: access_level <= 2 AND department_tag = user department.
Tier 3 Executive: broader cross-department and restricted summaries. Filter: access_level <= 3 AND role in (Manager, Executive).
Tier 4 System administrator: pipeline, audit logs, privilege assignment. Highly restricted data also needs an admin clearance token.
The filter is applied inside the SQL query (Layer 4), before the LLM sees any text, never afterwards.

Mapping Slack roles to tiers (proposal for the team to confirm)

Slack role is not used for access. The registry decides.
Team members (Owner): registered by the admin script. Project manager = tier 4; developers = tier 2 so they can test restricted pages.
Mr. Phat (regular member): tier 1, so the client sees what an ordinary employee would see.
Anyone not in user_registry, even an Owner: denied.
Project B swaps the registry lookup for VIB SSO; the tier map stays the same.

Hình 4. Chi tiết Lớp 2: ba cổng vào, đường từ chối (nét đỏ), cổng ra và audit.

Lớp 3 · RAG core (xử lý thông minh)

Vai trò: hiểu câu hỏi, lấy đúng những đoạn người đó được phép xem, rồi viết câu trả lời **bắt buộc phải dẫn nguồn**. Có 9 bước, chia thành ba nhóm việc: hiểu (1–3), quyết định (4–6) và viết (7–9).

Bước	Việc làm	Giải thích đơn giản
1. Nhận diện ngôn ngữ và làm sạch	Phát hiện tiếng Anh hay tiếng Việt, bỏ phần @mention, mở rộng viết tắt	Để trả lời đúng ngôn ngữ của người hỏi
2. Tra cache	Câu hỏi giống hoặc gần giống một câu vừa trả lời?	Trúng thì trả luôn, bỏ qua các bước tìm, nên rất nhanh
3. Router nhận biết vai trò	Chọn không gian cần tìm (HR, Deal Desk, Policies) và thêm bộ lọc space_key, department_tag, access_level ≤ tầng	Chỉ tìm trong ngăn tủ người này được mở
4. Retriever	Biến câu hỏi thành vector (embedding đa ngôn ngữ), gọi Lớp 4 lấy 50 đoạn gần nhất kèm bộ lọc	Tìm theo ý nghĩa, không chỉ theo từ khóa
5. Re-ranker (Beta)	Chấm điểm lại 50 đoạn bằng mô hình cross-encoder, giữ 3 đoạn tốt nhất	Lọc nhiễu để AI không bị rối
6. Kiểm tra độ liên quan	Có đoạn nào vượt ngưỡng điểm không? Không có thì trả lời dự phòng và dừng	Thà nói “không tìm thấy” còn hơn bịa
7. Dựng prompt	Luật hệ thống: chỉ trả lời từ ngữ cảnh, bắt buộc trích dẫn [1]..[n], trả lời theo ngôn ngữ câu hỏi, kèm 3 đoạn	Dặn AI cách làm bài
8. LLM engine	Project A: Ollama chạy Llama-3-8B hoặc Phi-3 trên máy, T = 0.1. Project B: Amazon Bedrock (Claude), ký bằng IAM	Đổi bằng cấu hình, không sửa code
9. Định dạng trích dẫn	Đổi source_uri thành link Confluence, loại trùng, xếp theo độ liên quan	Để Lớp 1 hiển thị thành nút bấm

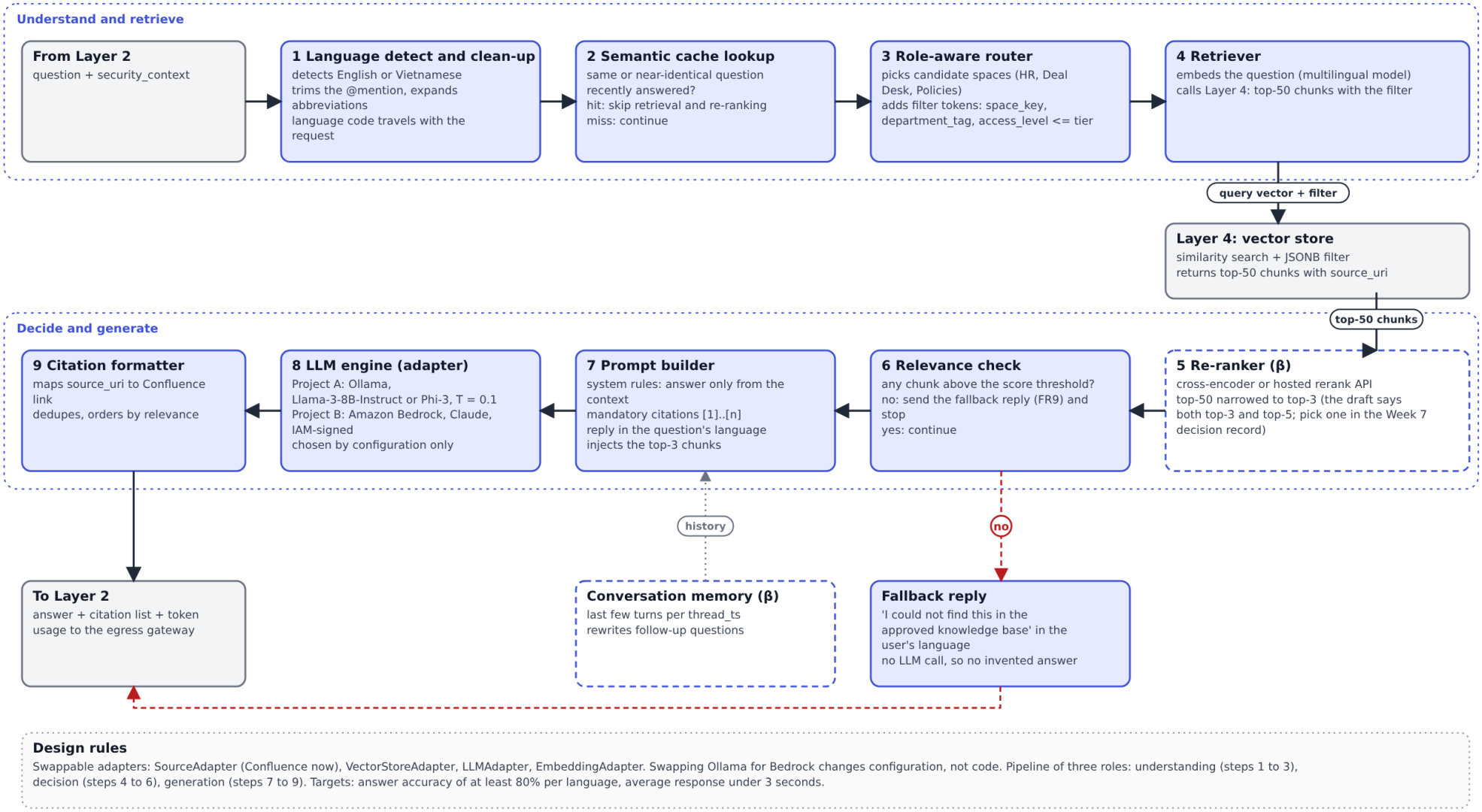
Nhiệt độ T = 0.1 nghĩa là AI được đặt ở chế độ “ít sáng tạo”, cho câu trả lời ổn định và bám sát tài liệu.

Quy tắc thiết kế: bộ chuyển đổi (adapter)
Nguồn tri thức (Confluence), kho vector, mô hình AI và mô hình embedding đều nằm sau các giao diện nhỏ (SourceAdapter, VectorStoreAdapter, LLMAdapter, EmbeddingAdapter). Nhờ vậy đổi Ollama sang Bedrock, hoặc thêm một nguồn dữ liệu mới, chỉ cần đổi cấu hình hoặc thêm một adapter, không viết lại lõi.

Bộ nhớ hội thoại (Beta): lưu vài lượt gần nhất của mỗi thread để hiểu câu hỏi nối tiếp như “thế còn trường hợp nghỉ ốm thì sao?”.

Layer 3: RAG core and business logic

Understand the question, retrieve only what the person may see, then generate an answer that must cite its sources. Every external piece sits behind an adapter.



Hình 5. Chi tiết Lớp 3: chín bước, nhánh trả lời dự phòng (nét đỏ) và bộ nhớ hội thoại (Beta).

Lớp 4 · Data ingestion and storage (dữ liệu)

Vai trò: biến các trang Confluence thành "kho tri thức" mà bot tìm được trong vài mili-giây, mỗi đoạn đều gắn nhãn ai được phép xem. Lớp này có hai chế độ: **chuẩn bị từ trước** (offline) và **tìm kiếm lúc có câu hỏi** (online).

A. Nạp dữ liệu (offline)

Bước	Việc làm	Công nghệ
A. Connector REST	Gọi Confluence REST API theo space_key, lấy nội dung, metadata và tệp đính kèm, có phân trang và thử lại khi bị giới hạn tốc độ	Python 3.11, API token trong biến môi trường
B. Raw staging	Lưu bản gốc (JSON, HTML) để chạy lại bộ phân tích mà không phải gọi lại Confluence. Project A: thư mục local /data/raw_dumps. Project B: S3 mã hóa SSE-S3	boto3 (Project B)
C. Bộ xử lý theo định dạng	Mỗi loại nội dung một bộ xử lý (xem bảng bên dưới)	BeautifulSoup4, html5lib, pandas, Unstructured
D–E. Chuẩn hóa và cắt đoạn	Làm sạch rồi cắt thành các đoạn 512 token, chồng nhau 50 token để không mất ý ở chỗ cắt	LangChain splitters
F. Gắn nhãn (metadata)	Mỗi đoạn được gắn: space_key, department_tag, file_type, access_level, source_uri, checksum_hash (MD5; trang đổi thì tính lại)	Pydantic, JSONB
G. Embedding	Biến mỗi đoạn thành một vector số. Dùng chung một mô hình đa ngôn ngữ Anh–Việt cho cả lúc nạp và lúc hỏi	Mô hình mã nguồn mở (A) hoặc Titan (B)
H. Lưu kho	Ghi hàng loạt (bulk insert/COPY), cập nhật theo checksum_hash	psycopg3, SQLAlchemy

Các bộ xử lý theo định dạng

Nội dung	Cách xử lý	Giai đoạn
Trang wiki HTML	Chuyển sang Markdown sạch, bỏ macro, giữ liên kết nội bộ	Alpha
Bảng phức tạp (ô gộp, lồng nhau)	Chuyển thành bảng Markdown hoặc JSON để con số không mất tiêu đề cột	Alpha
Ảnh nhúng	Lưu liên kết ảnh và chú thích cùng trang	Alpha
OCR ảnh chụp màn hình	Đọc chữ trong ảnh (pytesseract)	Beta
Tệp Word / PDF đính kèm	python-docx, pdfplumber, LibreOffice đổi .doc cũ sang .docx	Beta
Bảng tính Excel / CSV	openpyxl, pandas, mỗi dòng giữ tiêu đề cột	Beta
Web đối tác	Bộ lấy dữ liệu cắm thêm (Playwright), không đụng đến phần Confluence	Tương lai

B. Kho lưu trữ và tìm kiếm (online)

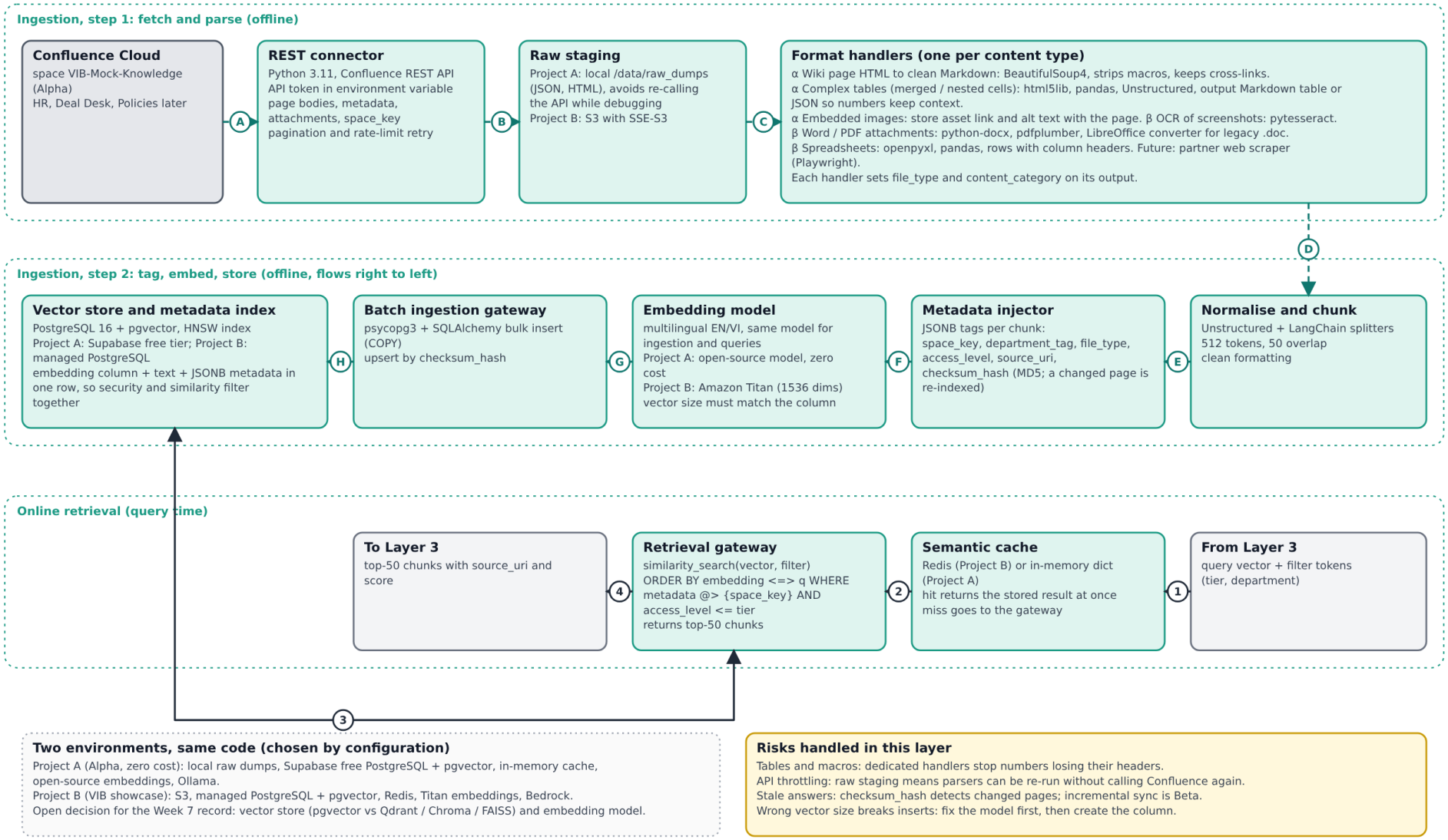
- **Kho vector:** PostgreSQL 16 với tiện ích pgvector, chỉ mục HNSW để so khớp tương đồng nhanh. Vector, văn bản và metadata nằm **cùng một dòng**, nên lọc quyền và tìm theo ý nghĩa diễn ra trong một truy vấn.

- **Semantic cache:** Redis (Project B) hoặc từ điển trong bộ nhớ (Project A) giữ kết quả của các câu hỏi thường gặp để giữ thời gian phản hồi.
- **Retrieval gateway:** nhận vector và bộ lọc từ Lớp 3, chạy similarity_search kèm điều kiện metadata và access_level ≤ tầng, trả về 50 đoạn kèm source_uri và điểm.

Hai môi trường, cùng một code (chọn bằng cấu hình)
Project A (Alpha, không tốn tiền): raw dump trên máy, PostgreSQL + pgvector miễn phí (Supabase free tier theo bản nháp), cache trong bộ nhớ, embedding mã nguồn mở, Ollama.
Project B (demo cho VIB): S3, PostgreSQL có quản lý + pgvector, Redis, embedding Titan, Bedrock.

Layer 4: Data ingestion and storage

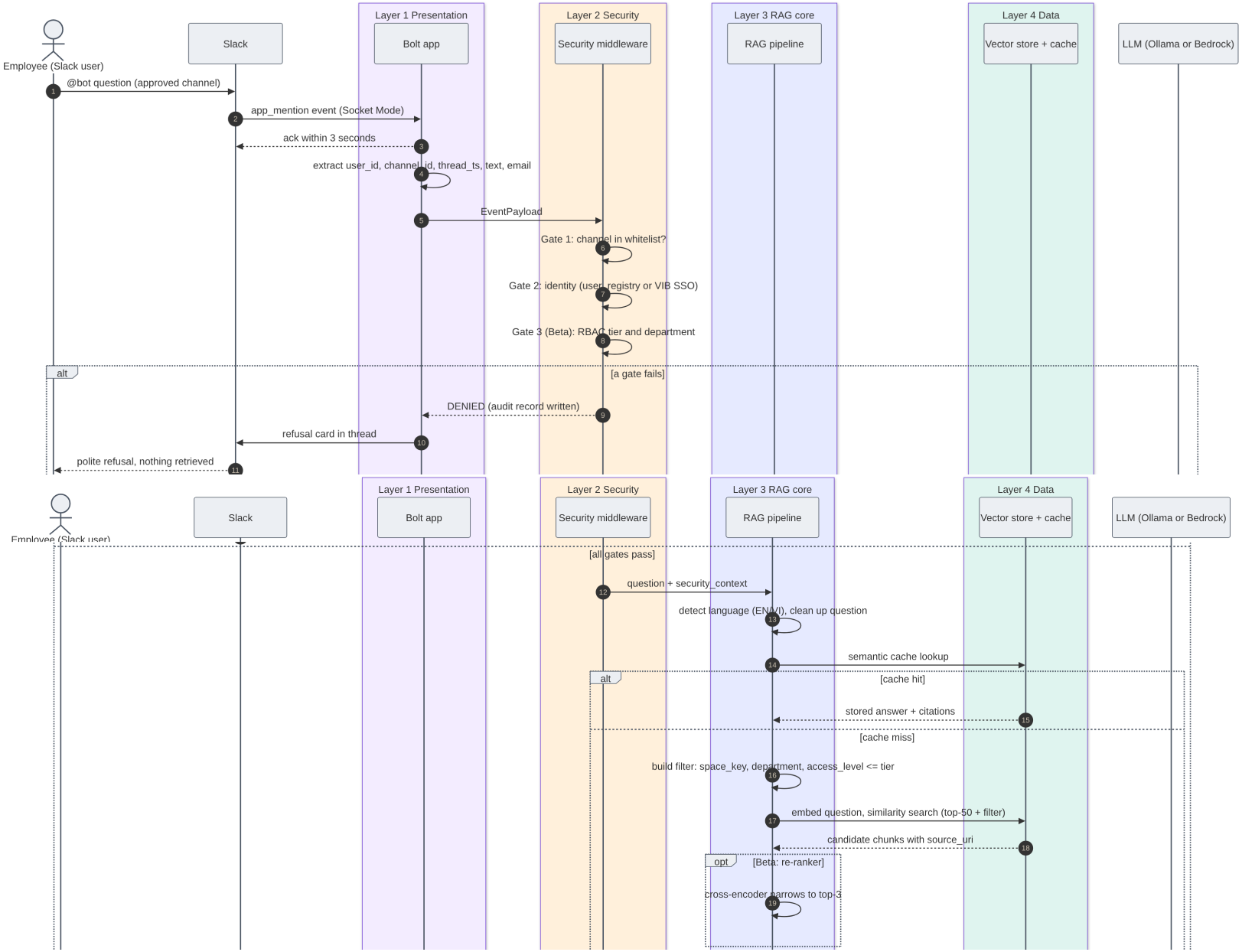
Offline ingestion (dashed, lettered) turns Confluence pages into tagged, embedded chunks. Online retrieval (solid) reads them back through a filtered query.

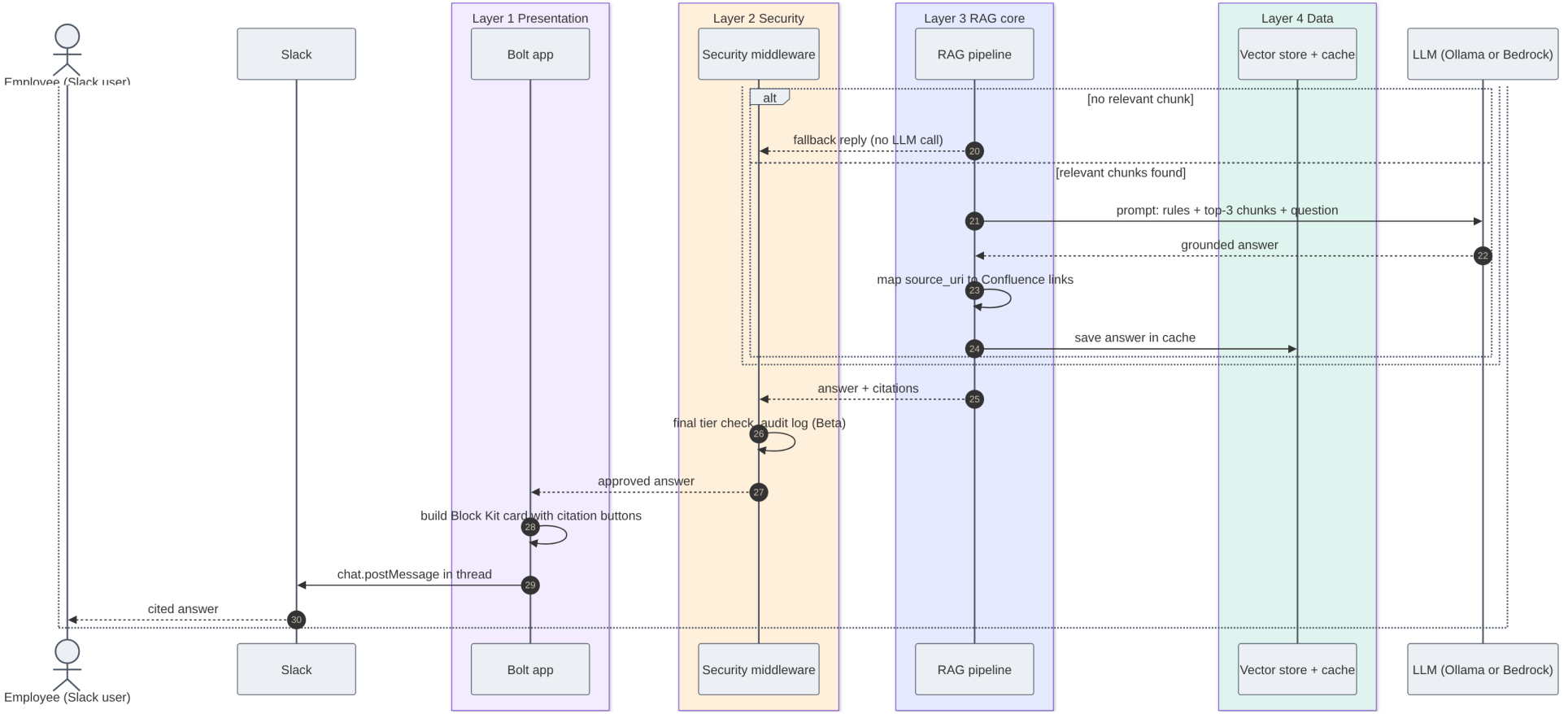


Hình 6. Chi tiết Lớp 4: hai bước nạp dữ liệu (A–H), phần tìm kiếm lúc có câu hỏi (1–4) và các rủi ro.

5. Một câu hỏi đi qua hệ thống từng bước (sequence diagram)

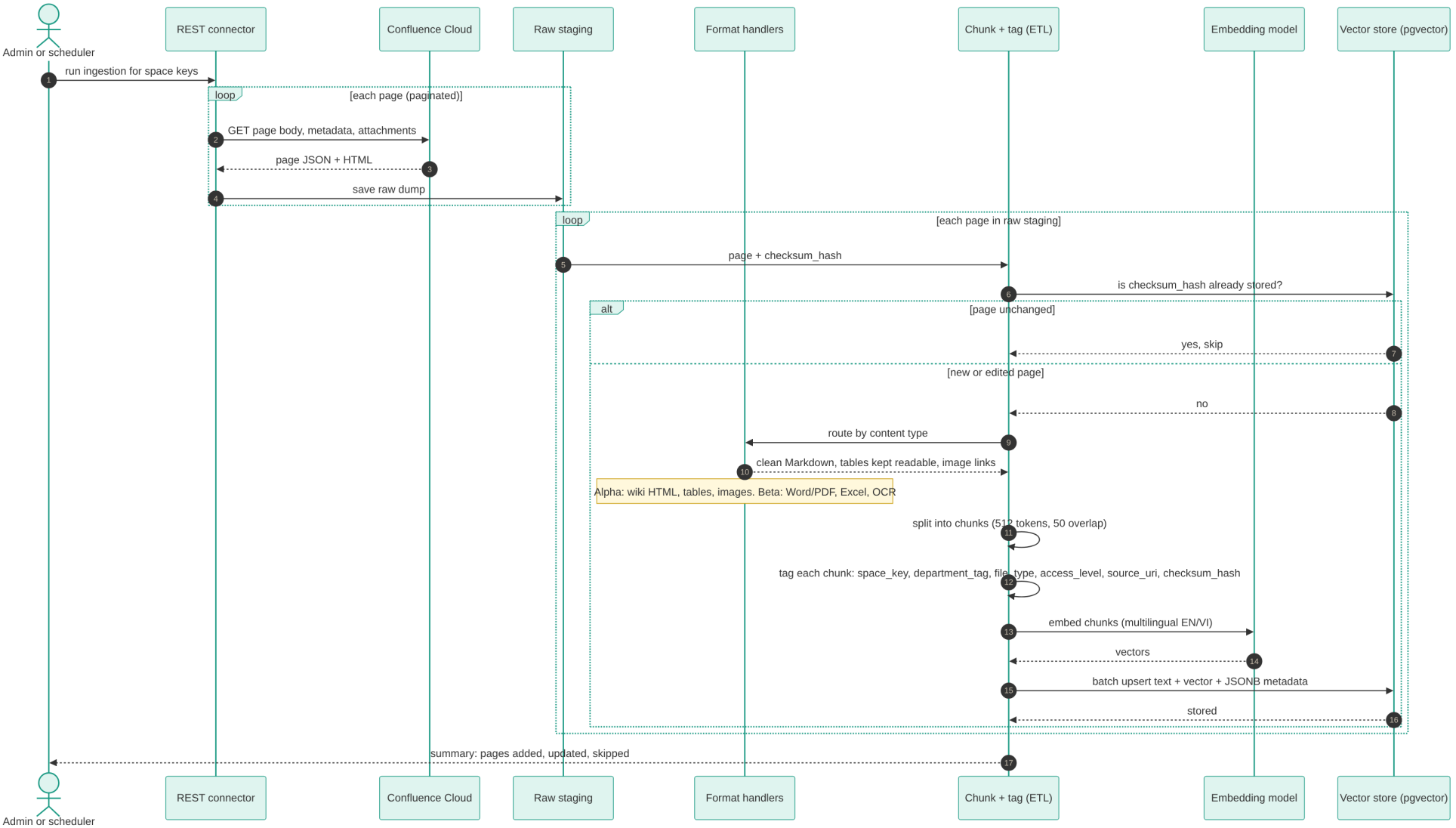
Ví dụ: nhân viên hỏi “Quy trình onboarding là gì?” trong một kênh đã được duyệt. Đọc từ trên xuống: mỗi mũi tên là một thông điệp, các khung **alt** là các trường hợp rẽ nhánh.





Hình 7. Sequence diagram luồng hỏi–đáp (3 phần liên tiếp). Có các nhánh: bị từ chối, trúng cache, không có đoạn liên quan (trả lời dự phòng).

Luồng nạp dữ liệu từ Confluence



Hình 8. Sequence diagram luồng nạp dữ liệu. Trang không đổi (checksum_hash đã có) thì được bỏ qua.

6. Alpha và Beta: cái gì làm trước

Charter chốt phạm vi Alpha. Bản nháp của nhóm có thêm vài mục thuộc Beta, nên cần quyết định rõ để không làm quá sức trong 12 tuần.

Thuộc Alpha (cần có)	Thuộc Beta / mở rộng
Nạp Confluence bằng REST, chuyển HTML sang Markdown, giữ bảng và ảnh	Re-ranker (cross-encoder)
Chia đoạn, embedding đa ngôn ngữ, lưu kho vector	Phân quyền theo tầng (RBAC) và lọc theo tài liệu
Slack app Bolt, Socket Mode, trả lời @mention trong thread	Audit log đầy đủ (Prometheus)
RAG có trích dẫn bắt buộc, nhận diện ngôn ngữ, trả lời dự phòng	Bộ nhớ hội thoại
Whitelist kênh (mặc định từ chối), xác thực người dùng (registry hoặc SSO)	Nút phản hồi hữu ích / chưa hữu ích
Block Kit, cache câu trả lời thường gặp, cấu hình ngoài code	Đồng bộ gia tăng khi trang thay đổi
Bàn giao: code, thiết kế, hướng dẫn triển khai, bằng chứng kiểm thử, đặc tả kỹ thuật	Nguồn bổ sung: Word/PDF, Excel, OCR, GitHub, web đối tác
	Kế hoạch sẵn sàng sản xuất (production)

Mẹo thực tế: dù RBAC thuộc Beta, vẫn nên **gắn access_level cho từng đoạn ngay lúc nạp dữ liệu**. Việc này gần như không tốn thêm công, và sau này bật phân quyền chỉ cần thêm bộ lọc (đề xuất).

7. Những điểm nhóm cần chốt

#	Vấn đề	Tình trạng và đề xuất
1	Số đoạn đưa vào AI	Bản nháp ghi cả top-3 lẫn top-5. Hình vẽ dùng **top-3**. Cần chọn một con số và ghi vào decision record tuần 7
2	Mô hình embedding và kích thước vector	Bản nháp dùng Amazon Titan (1536 chiều), có phí, không khớp mục tiêu Alpha 0 đồng. Đề xuất dùng mô hình đa ngôn ngữ mã nguồn mở cho Alpha và tạo cột vector đúng kích thước của mô hình đã chọn
3	Kho vector	Còn mở: pgvector hay Qdrant / Chroma / FAISS. Quy tắc chọn: stack nhỏ nhất đạt mục tiêu chính xác và dưới 3 giây, chi phí 0, có lọc metadata để phân quyền
4	Mô hình AI	Còn mở: Ollama chạy trên máy hay API được duyệt (ví dụ Amazon Bedrock). Bản nháp: Ollama cho Project A, Bedrock cho Project B
5	Xác thực phiên bản B	Nhóm cần xác nhận với anh Phát: dùng SSO của VIB và kiểm tra tên miền @vib.com.vn?
6	Chi phí Slack và Confluence	Biên bản họp 22/10 ghi chưa quyết định gói trả phí. Cần anh Phát xác nhận gói miễn phí có đủ cho thử nghiệm
7	Ánh xạ role sang tầng quyền	Đề xuất ở Lớp 2: PM tầng 4, dev tầng 2, anh Phát tầng 1. Cần cả nhóm đồng ý

8. Câu hỏi thường gặp khi bảo vệ

Hỏi: Vì sao dùng RAG mà không huấn luyện lại (fine-tune) mô hình?

Đáp: Tài liệu thay đổi liên tục nên mỗi lần đổi phải huấn luyện lại, rất tốn kém. Fine-tune cũng không cho trích dẫn chính xác trang nguồn và không phân quyền theo từng tài liệu. RAG chỉ cần nạp lại các trang đã đổi.

Hỏi: Làm sao chắc bot không nói bịa?

Đáp: Ba lớp chặn: prompt buộc AI chỉ dùng ngữ cảnh được cấp, bắt buộc trích dẫn nguồn, và nếu không có đoạn nào đủ liên quan thì trả lời dự phòng mà không gọi AI. Độ chính xác được chấm trên bộ câu hỏi song ngữ bằng thang đạt / một phần / không đạt, do hai người chấm độc lập.

Hỏi: Làm sao ngăn người không có quyền xem được tài liệu nhạy cảm?

Đáp: Quyền được lọc ngay trong truy vấn cơ sở dữ liệu, trước khi AI nhìn thấy văn bản, và kiểm tra lại ở cổng ra. Kênh và người dùng chưa duyệt không nhận được gì.

Hỏi: Chạy mà không tốn tiền bằng cách nào?

Đáp: Ollama chạy mô hình trên máy, cơ sở dữ liệu miễn phí (Supabase free tier), cache trong bộ nhớ, Socket Mode không cần hạ tầng công khai. Chi phí vận hành thật ước tính khoảng 50–85 USD mỗi tháng (hosting, kho vector, token) nhưng nằm ngoài phạm vi PoC.

Hỏi: Đo độ chính xác và tốc độ như thế nào?

Đáp: Độ chính xác: bộ câu hỏi song ngữ, chấm theo checklist, mục tiêu từ 80% trở lên cho mỗi ngôn ngữ. Tốc độ: báo cáo trung bình, trung vị và phân vị 95 của thời gian từ lúc hỏi đến lúc có câu trả lời, mục tiêu trung bình dưới 3 giây.

9. Bảng thuật ngữ

Thuật ngữ	Nghĩa đơn giản
RAG	Tìm tài liệu liên quan trước, rồi mới viết câu trả lời dựa trên đó
LLM	Mô hình ngôn ngữ lớn, phần “AI” viết câu trả lời
Embedding	Biến một đoạn văn thành dãy số mô tả ý nghĩa, để máy so sánh “gần nghĩa”
Vector store / pgvector	Kho lưu các dãy số đó và tìm đoạn gần nghĩa nhất với câu hỏi
Chunk	Một đoạn nhỏ của trang (512 token), là đơn vị được lưu và tìm
Metadata (JSONB)	Thông tin gắn kèm mỗi đoạn: thuộc không gian nào, phòng ban nào, mức truy cập nào, từ trang nào
Socket Mode	Cách bot tự kết nối ra Slack bằng WebSocket, không cần địa chỉ công khai
Whitelist	Danh sách kênh được phép. Không có trong danh sách thì bị bỏ qua
SSO	Đăng nhập một lần bằng tài khoản công ty
RBAC	Phân quyền theo vai trò: vai trò quyết định xem được tài liệu nào
Re-ranker	Bước chấm điểm lại để chọn ra vài đoạn liên quan nhất
Block Kit	Cách Slack dựng thẻ tin nhắn có nút bấm và phần nguồn
Citation	Trích dẫn: link về đúng trang Confluence đã dùng
Fallback	Câu trả lời dự phòng khi không tìm thấy thông tin phù hợp
Adapter	Giao diện trung gian để đổi mô hình AI, kho dữ liệu mà không sửa lõi

Nguồn: Project Charter, Project Proposal, Project Team and Project Plan, biên bản họp 22/10/2026, bản nháp draw.io của nhóm (5 trang), README của repo Slack_Bot_Project_Brok. Các mục ghi là “đề xuất” là ý kiến của người soạn tài liệu này, chưa phải quyết định của nhóm.